

MEDELLIN LOUNGE

Système de Gestion Interne — Lounge Chicha & Boissons — Conakry, Guinée

Cahier des Charges — v1.2

Champ	Valeur
Version	v1.2
Dépôt	https://github.com/jeunesavane-ctrl/Monprojet.git (branche <code>main</code>)
Production	https://medellin-lounge.com (Netlify Pro — déploiement <code>manuel</code>)
Supabase	https://xkdlkvwtzfixsaiedkf.supabase.co
Local	<code>python -m http.server 5500</code> → http://localhost:5500
Propriétaire	Mohamed Lamine Savane — NEXUS SERVICES & CONSTRUCTION SARL

NEXUS SERVICES & CONSTRUCTION SARL — *Confidentiel*

1. Vision & Philosophie

« Un seul franc qui entre dans le système doit agir partout, être compté partout, ne jamais passer inaperçu. »

Medellin Lounge est un lounge chicha et boissons à Conakry, Guinée. Le système gère les ventes, la caisse, les achats, le stock chicha, les ressources humaines et la distribution des bénéfices aux associés.

Mobile-first : le personnel travaille sur téléphone. Chaque écran est conçu pour un usage tactile, en condition réelle de service.

Principe directeur — traçabilité totale. Toute somme est rattachée à une personne, une session et un horodatage. Le chiffre d'affaires est **dérivé des ventes enregistrées** (jamais ressaisi librement), le contrôle de caisse compare le théorique au réel, et tout écart est attribuable à une personne.

2. Acteurs, Rôles & Authentification

2.1 Les trois catégories d'acteurs — emplacements séparés

Trois catégories d'acteurs, stockées dans **trois emplacements distincts**. Les associés (co-investisseurs) et les employés (salariés) ne sont **jamais** mélangés.

Catégorie	Emplacement Supabase	Rôle(s)	Colonnes clés
Gestionnaire	<code>config</code>	<code>owner</code>	<code>pin_owner</code> , <code>owner_nom</code> , <code>owner_pct</code>
Associés	<code>associes</code>	<code>associe</code>	<code>id</code> , <code>nom</code> , <code>prenom</code> , <code>pourcentage</code> NUMERIC, <code>pin_hash</code> , <code>actif</code>
Employés	<code>employes</code>	<code>manager</code> / <code>caissier</code> / <code>staff</code> / <code>chicha</code> / <code>achats</code>	<code>id</code> , <code>nom</code> , <code>prenom</code> , <code>poste</code> , <code>role</code> , <code> salaire_base</code> INT, <code>pin_hash</code> , <code>actif</code>

- `employes` = **salariés uniquement**. Pas de colonne `pourcentage` .
- `associes` = **co-investisseurs uniquement**. Ils touchent des parts, pas de salaire.
- **Le manager est un employé à part entière** (`employes` , `role=manager`) : il est suivi en paie, en présences et en avances comme tout salarié.

2.2 Flux d'authentification — ordre strict dans `index.html`

L'ordre de vérification du PIN ne doit jamais dévier :

```
1. sha256(PIN) == config.pin_owner      → role="owner"      extra={nom: config.owner_nom}
2. sha256(PIN) == config.pin_manager    → role="manager"    extra={nom: "Manager", acces: "secours"} (sans employe_id)
3. sha256(PIN) == employes.pin_hash     → role=emp.role     extra={nom, employe_id}
4. sha256(PIN) == associes.pin_hash     → role="associe"    extra={nom, associe_id, pourcentage}
5. aucune correspondance                → message d'erreur, pas de connexion
```

- `config.pin_manager` est un accès de secours / d'amorçage uniquement. Il ouvre une session `manager` sans `employe_id` (donc non suivie en paie). En exploitation normale, chaque manager possède son propre enregistrement dans `employes` (étape 3) et se connecte avec son PIN individuel.
- `sessionStorage` : `ml_role`, `ml_hashes` `{hOwner, hManager}`, `ml_extra`.
- **Auto-lock** : 10 minutes d'inactivité → `ML.lock()`.
- Le système utilise les **PINs individuels** (`employes.pin_hash` et `associes.pin_hash`). La clé `pin_staff` de `config` n'intervient pas dans le flux d'authentification.

2.3 Clés de configuration (`config`)

Clé	Description	Défaut
<code>pin_owner</code>	Hash SHA-256 du PIN gestionnaire	— (à définir)
<code>pin_manager</code>	Hash SHA-256 du PIN manager de secours	— (à définir, à changer)
<code>pin_staff</code>	Hash SHA-256 PIN staff global (non utilisé en auth)	— (à changer)
<code>owner_nom</code>	Nom affiché du gestionnaire	—
<code>owner_pct</code>	% du gestionnaire dans la distribution	<code>100 - SUM(associes.pourcentage actifs)</code>
<code>part_lounge</code>	% du net réservé au lounge avant distribution	<code>10</code>
<code>objectif_journalier</code>	Objectif de net par jour (GNF)	—
<code>message_manager</code>	Message global affiché aux managers	—

Invariant des parts. `owner_pct + SUM(associes.pourcentage actifs)` doit toujours valoir **100**. `parametres.html` refuse l'enregistrement si la somme est différente de 100. À ne pas confondre avec `part_lounge`, qui est prélevé **avant** la répartition entre owner et associés.

3. Navigation — 18 pages

Chaque page filtre son accès par rôle via `initPage([ ... ])`.

Page	Rôles autorisés	Responsabilité
<code>dashboard.html</code>	manager / owner / associe	KPIs, graphiques, sessions en attente, validation manager
<code>saissie.html</code>	staff	Saisie des ventes — sélection de table obligatoire — multi-tours — verrou session
<code>chicha.html</code>	chicha / manager / owner	Sorties chicha — inventaire stock uniquement , jamais financier
<code>achats.html</code>	achats / manager / owner	Saisie des achats et dépenses dans la session caisse
<code>caisse.html</code>	caissier / manager / owner / associe*	Session caisse, <code>verifications_staff</code> par serveuse, écarts, clôture (*associé : lecture seule)
<code>rapport.html</code>	manager / owner	Rapport journalier — CA pré-rempli depuis les ventes, ajustable
<code>pointage.html</code>	manager / owner	Raccourci vers <code>rh.html > Présences</code>
<code>historique.html</code>	manager / owner	Rapports avec filtres, suppression (owner), export CSV
<code>fiche.html</code>	staff / caissier / chicha / achats / manager	Solde salaire, avances, écarts personnels (lecture seule)
<code>avance.html</code>	staff / caissier / chicha / achats / manager	Demande d'avance sur salaire
<code>produits.html</code>	manager / owner	Catalogue produits et gestion du stock
<code>rh.html</code>	manager / owner	5 onglets : Équipe / Présences / Demandes / Avances / Paie
<code>avances.html</code>	manager / owner	Raccourci vers <code>rh.html > Demandes / Avances</code>
<code>charges.html</code>	manager / owner	Charges fixes mensuelles (jamais les salaires)
<code>finances.html</code>	owner / manager / associe	Bilan, dividendes, trésorerie, charges, évolution — onglets selon rôle
<code>bilan.html</code>	owner / manager / associe	Bilan mensuel ; l'associé ne voit que sa propre part
<code>associes.html</code>	owner / associe	Gestion des associés (owner : % + PIN) ; l'associé voit sa propre participation
<code>parametres.html</code>	owner	PINs, configuration générale, gestion des tables du lounge

Pages-raccourcis. `pointage.html` et `avances.html` n'ont **pas** de logique propre : elles redirigent vers les onglets correspondants de `rh.html`, qui est la **seule autorité** pour les présences et les avances. Elles existent comme entrées de navigation rapides.

Badges de navigation

Badge	Condition déclenchante	Visible par
<code>nbadge-caisse</code>	<code>remboursements_ecart</code> en statut <code>en_attente</code>	caissier, manager, owner
<code>nbadge-avances</code>	<code>avances</code> en statut <code>en_attente</code>	manager, owner

4. Base de Données — 22 tables

Impératif. Ces noms de colonnes sont définitifs. Utiliser **exactement** ces noms — jamais de variantes (`libelle`, `prix`, `seuil_bas`, `table_num` ...).

Table	Colonnes clés	Notes
config	key TEXT UNIQUE, value TEXT	Clés : pin_owner , pin_manager , pin_staff , owner_nom , owner_pct , part_lounge (défaut 10), objectif_journalier , message_manager
associes	id UUID PK, nom , prenom , pourcentage NUMERIC, pin_hash TEXT, actif BOOLEAN	Co-investisseurs, séparés des employés. Gérés dans associes.html (% + PIN, invariant owner+associés=100%).
employees	id UUID PK, nom , prenom , poste , role TEXT, salaire_base INT, pin_hash TEXT, actif BOOLEAN	Salariés uniquement (manager inclus). role : manager / caissier / staff / chicha / achats. Pas de colonne pourcentage
logs	id UUID PK, role TEXT, action TEXT, timestamp TIMESTAMPTZ	Toutes les actions via ML.logAction()
produits	id UUID PK, nom , type (chicha/boisson/autre), stock_actuel NUMERIC, stock_min NUMERIC, prix_default INT, prix_achat INT, unite_vente , packaging_label , unite_par_packaging NUMERIC, actif BOOLEAN NOT NULL DEFAULT true	prix_default (pas prix). stock_min (pas seuil_bas). Filtre actifs : .not("actif", "is", false) . Voir \$4c pour le stock
tables_lounge	id UUID PK, label TEXT UNIQUE, ordre INT, actif BOOLEAN, ouverte_par UUID REFERENCES employees(id) DEFAULT NULL	ouverte_par = serveuse qui a activé la table en premier (affichage seulement). NULL = libre. Remis à NULL après valide_manager
sessions_caisse	id UUID PK, date DATE UNIQUE NOT NULL, statut TEXT, fond_caisse INT, total_reel INT, total_om_verifie INT, ecart_especes INT, ecart_om INT, ecart INT, surplus_caisse INT, caissier_id UUID, note_caissier TEXT, note_manager TEXT	UNIQUE(date). statut : ouvert / valide_caissier / valide_manager . 3 colonnes d'écart distinctes. caissier_id NULL à la création, obligatoire à la clôture (garde applicative)
ventes_session	id UUID PK, session_id UUID, employe_id UUID, produit_id UUID, produit_nom TEXT, qty INT, prix_unitaire INT, total INT, paiement TEXT (especes / om), table_label TEXT	table_label = label TEXT de la table. Plusieurs serveuses peuvent avoir des lignes sur le même table_label . Pas de colonne table_num .
verifications_staff	id UUID PK, session_id UUID, employe_id UUID, recu_especes INT, recu_om INT — UNIQUE(session_id , employe_id)	Argent réellement reçu par la caissière de chaque serveuse. Base du calcul des écarts. UPSERT sur (session_id , employe_id)
mouvements_caisse	id UUID PK, session_id UUID, type TEXT CHECK (entree/sortie), motif TEXT, montant INT > 0, note TEXT, created_at	Entrées / sorties de caisse hors ventes
achats_session	id UUID PK, session_id UUID, categorie TEXT, produit_nom TEXT, montant INT > 0, qty NUMERIC, prix_unitaire INT, created_at	Source unique des achats du jour. Pré-remplit rapports.total_achats
sorties_chicha	id UUID PK, session_id UUID, employe_id UUID, arome TEXT, qty INT, valide BOOLEAN, created_at	Stock inventaire uniquement — jamais dans les calculs financiers
rapports	id UUID PK, date DATE UNIQUE, num INT NOT NULL, session_id UUID, total_chicha INT, total_boissons INT, total_achats INT, recettes INT, net INT, manager TEXT, caissier TEXT, part JSONB, chicha_rows JSONB, boissons_rows JSONB, achats_rows JSONB	UNIQUE(date). num = entier MAX+1 . recettes = total_chicha + total_boissons . net = recettes - total_achats
presences	id UUID PK, employe_id UUID, date DATE, statut TEXT (present/absent/retard/conge), note — UNIQUE(employe_id , date)	4 statuts. Saisi dans rh.html > Présences
justifications	id UUID PK, employe_id UUID, date DATE, type TEXT (absence / retard / autre), motif TEXT, statut TEXT (en_attente/approuvee/rejetee)	Seules les justifications type="absence" et statut="approuvee" réduisent absNJ
avances	id UUID PK, employe_id UUID, montant INT > 0, date DATE, statut TEXT (en_attente/approuvee/rejetee), remboursee BOOLEAN DEFAULT false, note_demande TEXT, obs TEXT	note_demande = note du demandeur ; obs = observation du valideur. Déduites si statut=approuvee AND remboursee=false , toutes dates
salaires_verses	id UUID PK, employe_id UUID, mois TEXT (YYYY-MM), salaire_brut INT, avances_deduites INT, ecarts_deduits	Voir règles VERSER / ANNULER (\$7.2)

Table	Colonnes clés	Notes
	INT, <code>surplus_caisse</code> INT, <code>net_verse</code> INT, <code>nb_absences_nj</code> INT, <code>sanction_type</code> TEXT, <code>sanction_montant</code> INT, <code>nb_retards</code> INT, <code>sanction_retard_montant</code> INT, <code>paye_le</code> DATE — UNIQUE(<code>employe_id</code> , <code>mois</code>)	
<code>charges</code>	<code>id</code> UUID PK, <code>label</code> TEXT, <code>montant</code> INT > 0, <code>mois</code> TEXT (YYYY-MM), <code>categorie</code> TEXT, <code>paye</code> BOOLEAN, <code>date_paieement</code> DATE, <code>recurrence</code> TEXT	<code>label</code> (pas <code>libelle</code>). Ne contient jamais de salaires (ceux-ci vivent dans <code>salaires_verses</code>)
<code>remboursements_ecart</code>	<code>id</code> UUID PK, <code>session_id</code> UUID, <code>employe_id</code> UUID, <code>montant</code> INT > 0, <code>note</code> TEXT, <code>statut</code> TEXT (en_attente/valide/rejete), <code>created_at</code>	Badge rouge nav caisse quand <code>en_attente</code>
<code>credits</code>	<code>id</code> UUID PK, <code>employe_id</code> UUID, <code>session_id</code> UUID, <code>montant</code> INT > 0, <code>rembourse</code> BOOLEAN DEFAULT false	Table conservée mais non utilisée — crédit retiré (pas d'ardoises).
<code>propositions</code>	<code>id</code> UUID PK, <code>titre</code> TEXT, <code>description</code> TEXT, <code>auteur_nom</code> TEXT, <code>statut</code> TEXT (ouvert/ferme), <code>created_at</code>	Table conservée mais non utilisée (propositions/votes retirés)
<code>votes_prop</code>	<code>id</code> UUID PK, <code>proposition_id</code> UUID, <code>votant_key</code> TEXT, <code>votant_nom</code> TEXT, <code>poids</code> NUMERIC, <code>choix</code> BOOLEAN — UNIQUE(<code>proposition_id</code> , <code>votant_key</code>)	Table conservée mais non utilisée (votes retirés)

4b. Schéma SQL — création des objets

À exécuter dans Supabase SQL Editor **avant** tout développement.

```

-- 1. TABLE associes (co-investisseurs, séparée des employés)
CREATE TABLE IF NOT EXISTS associes (
  id UUID DEFAULT gen_random_uuid() PRIMARY KEY,
  nom TEXT NOT NULL,
  prenom TEXT,
  pourcentage NUMERIC NOT NULL DEFAULT 0,
  pin_hash TEXT NOT NULL,
  actif BOOLEAN NOT NULL DEFAULT true,
  created_at TIMESTAMPTZ DEFAULT now()
);
ALTER TABLE associes DISABLE ROW LEVEL SECURITY;

-- 2. TABLE tables_lounge
CREATE TABLE IF NOT EXISTS tables_lounge (
  id UUID DEFAULT gen_random_uuid() PRIMARY KEY,
  label TEXT NOT NULL UNIQUE,
  ordre INT NOT NULL DEFAULT 0,
  actif BOOLEAN NOT NULL DEFAULT true,
  ouverte_par UUID REFERENCES employes(id) DEFAULT NULL,
  created_at TIMESTAMPTZ DEFAULT now()
);
ALTER TABLE tables_lounge DISABLE ROW LEVEL SECURITY;
-- Renseigner les tables réelles du lounge depuis parametres.html (label + ordre).

-- 3. TABLE verifications_staff
CREATE TABLE IF NOT EXISTS verifications_staff (
  id UUID DEFAULT gen_random_uuid() PRIMARY KEY,
  session_id UUID NOT NULL REFERENCES sessions_caisse(id),
  employe_id UUID NOT NULL REFERENCES employes(id),
  recu_especes INT NOT NULL DEFAULT 0,
  recu_om INT NOT NULL DEFAULT 0,
  created_at TIMESTAMPTZ DEFAULT now(),
  UNIQUE(session_id, employe_id)
);
ALTER TABLE verifications_staff DISABLE ROW LEVEL SECURITY;

-- 4. Colonnes ajoutées aux tables existantes
ALTER TABLE sessions_caisse
  ADD COLUMN IF NOT EXISTS ecart_om INT DEFAULT 0;
ALTER TABLE sessions_caisse
  ADD COLUMN IF NOT EXISTS surplus_caisse INT DEFAULT 0;

ALTER TABLE ventes_session
  ADD COLUMN IF NOT EXISTS table_label TEXT;
CREATE INDEX IF NOT EXISTS idx_ventes_table_label
  ON ventes_session(table_label);

-- 5. Intégrité : un seul rapport par date (index unique idempotent)
CREATE UNIQUE INDEX IF NOT EXISTS rapports_date_unique ON rapports(date);

```

4c. Gestion du stock (`produits`)

- Une vente (`ventes_session`) **décrémente** `produits.stock_actuel` de la quantité vendue.
- Une sortie chicha (`sorties_chicha`) décrémente le stock de l'arôme concerné — **stock uniquement**.
- Un achat / réassort **incrémente** `stock_actuel` (`unite_par_packaging` convertit un packaging acheté — ex. carton — en unités de vente).
- `produits.html` signale visuellement tout produit dont `stock_actuel ≤ stock_min`.
- **Le stock n'entre jamais dans les calculs financiers** (P&L, écarts, distribution).

5. Flux Opérationnel Quotidien

Matin

- Le manager ou la caissière saisit le fond de caisse → session créée (`statut: ouvert`).
- Si deux rôles créent au même moment → re-SELECT si l'INSERT échoue (contrainte UNIQUE `date`).

Journée

- `saisie.html` (**staff**)
- Grille des tables (libre / active-moi / active-autre / verrouillée).
- La serveuse sélectionne une table → sélectionne les produits → mode de paiement (`especes` / `om`).
- `INSERT ventes_session (table_label, employe_id, produit_nom, total, paiement...)` + décrémente le stock.
- Si table libre : `UPDATE tables_lounge SET ouverte_par = employe_id`.
- Si table active : ventes ajoutées, `ouverte_par` inchangé.
- Multi-tours possibles dans la journée.
- Verrou lecture seule si `statut ≥ valide_caissier`.
- `chicha.html` (**chicha**) → `INSERT sorties_chicha` (stock uniquement, jamais financier).
- `achats.html` (**achats**) → `INSERT achats_session`.

Soir — Clôture (caissière) dans `caisse.html`

- Voit les ventes groupées par serveuse (et par table dans chaque carte serveuse).
- Pour chaque serveuse : saisit `recu_especes` + `recu_om` → `UPSET verifications_staff`.
- L'app affiche l'écart de chaque serveuse en temps réel.
- Compte le tiroir espèces → `total_reel`. Vérifie l'OM sur téléphone → `total_om_verifie`.
- L'app calcule `ecart_especes`, `ecart_om`, `ecart`, `surplus_caisse` → stockés dans `sessions_caisse`.
- **caissier_id est obligatoire**. Si l'owner ou un manager clôture à la place de la caissière, il sélectionne la caissière responsable.
- Clôture → `statut: valide_caissier`.

Soir — Validation (manager) dans `rapport.html` ou `dashboard.html`

- `total_chicha` / `total_boissons` sont **pré-remplis automatiquement** depuis `ventes_session` (regroupés par `produits.type`). Le manager peut ajuster ; l'app affiche l'écart `saisi - calculé`.
- `total_achats = SUM(achats_session.montant)` — **non modifiable**.
- `INSERT rapports`.
- `UPDATE sessions_caisse SET statut = valide_manager`.
- `UPDATE tables_lounge SET ouverte_par = NULL WHERE actif = true`.

Automatique

- Listener Supabase Realtime sur `rapports` → reload des KPIs du dashboard.

5.1 Logique des Tables

Principe. Une table appartient à la **session**, pas à une serveuse. La première serveuse qui active une table en devient la responsable principale (affichage seulement). N'importe quelle autre serveuse peut y ajouter des ventes. Les écarts restent **nominatifs par serveuse**.

Scénario	Comportement attendu
Une serveuse active une table libre	<code>INSERT vente (table_label , employe_id)</code> . <code>UPDATE tables_lounge SET ouverte_par = employe_id WHERE label = ...</code> . La table devient active
Une autre serveuse ajoute une commande sur la table active	<code>INSERT vente</code> (même <code>table_label</code> , autre <code>employe_id</code>). <code>ouverte_par</code> inchangé. Les deux serveuses ont des lignes sur cette table
Vue caissière sur une table partagée	Les lignes de chaque serveuse restent distinctes par <code>employe_id</code> ; chacune est vérifiée séparément
Fin de journée, après <code>valide_manager</code>	L'écart est calculé par serveuse (via <code>verifications_staff</code>), jamais par table
Reset quotidien	<code>UPDATE tables_lounge SET ouverte_par = NULL WHERE actif = true</code> . Toutes les tables repartent libres le lendemain
Fermeture forcée d'une table	Owner/manager : <code>UPDATE tables_lounge SET ouverte_par = NULL WHERE label = ...</code> . Les serveuses ne ferment pas les tables — elles clôturent leur session

5.2 États visuels des tables dans `saisie.html`

État	Apparence	Signification	Action possible
Libre	Fond sombre, label blanc	Aucune vente sur cette table dans la session	Toute serveuse peut l'activer
Active — moi	Bordure or pleine, sous-titre = mon prénom	J'ai déjà des ventes sur cette table	Peut ajouter des ventes
Active — autre	Bordure or subtile, prénom de l'ouvrante	Une autre serveuse a ouvert cette table	Peut quand même ajouter des ventes
Verrouillée	Icône cadenas, non cliquable	Session <code>statut ≥ valide_caissier</code>	Lecture seule

`ouverte_par` est remis à NULL automatiquement après validation manager. Aucune action manuelle requise en fin de journée.

6. Modèle des Écarts

Base de calcul. La référence de la caissière est ce qu'elle déclare avoir **réellement reçu** de chaque serveuse (`verifications_staff`), pas directement les ventes. La caissière compte ce qu'elle a en main, pas ce que les ventes annoncent.

6.1 Écart par serveuse — calculé en JS, jamais stocké en base

`recu_especes` / `recu_om` (`verifications_staff`) = argent **physiquement remis** par la serveuse à la caissière. `decl_esp` / `decl_om` = ce que les ventes (espèces + OM) **annoncent**.

```
-- Pour chaque serveuse X dans la session Y :
decl_esp = SUM(ventes_session.total WHERE paiement="especes" AND employe_id=X AND session_id=Y)
decl_om  = SUM(ventes_session.total WHERE paiement="om"      AND employe_id=X AND session_id=Y)
recu_esp = verifications_staff.recu_especes WHERE employe_id=X AND session_id=Y
recu_om  = verifications_staff.recu_om      WHERE employe_id=X AND session_id=Y

ecart_serveuse = (decl_esp + decl_om) - (recu_esp + recu_om)
  + → dette serveuse (elle a remis moins que prévu)
  - → excédent (elle a remis plus que prévu)
```

Ne pas stocker en base — calculé et affiché en JS dans `caisse.html`.

6.2 Écart caissière — stocké dans `sessions_caisse` à la clôture

```
-- Calcul dans caisse.html → validateSession() :
tot_recu_esp = SUM(verifications_staff.recu_especes) pour toute la session
tot_recu_om  = SUM(verifications_staff.recu_om)    pour toute la session
entrees      = SUM(mouvements_caisse.montant WHERE type="entree")
sorties      = SUM(mouvements_caisse.montant WHERE type="sortie")
achats       = SUM(achats_session.montant)

theoriqueEsp = fond_caisse + tot_recu_esp + entrees - sorties - achats
ecart_especes = theoriqueEsp - total_reel      → sessions_caisse.ecart_especes
ecart_om      = tot_recu_om - total_om_verifie → sessions_caisse.ecart_om
ecart         = ecart_especes + ecart_om       → sessions_caisse.ecart
surplus_caisse = MAX(0, -ecart)                → sessions_caisse.surplus_caisse
  + ecart → manque (déduit du salaire caissière)
  - ecart → surplus (bonus salaire caissière, via surplus_caisse)
```

- **Paie** : l'**écart total** (`ecart_especes + ecart_om`) impacte le salaire de la caissière. L'OM est de l'argent réel — le téléphone Orange Money est entre ses mains —, il compte donc **comme les espèces**.

6.3 Source de l'écart selon le rôle

Pour les deux rôles, on sépare la **dette** (écart positif → déduction) du **surplus** (écart négatif → bonus), toujours exprimés en valeur positive.

Rôle	Source	Déduction (dette)	Bonus (surplus)
Serveuse	<code>ecart_serveuse</code> via <code>verif_staff</code> (en JS)	$\Sigma \text{MAX}(0, \text{ecart_serveuse})$ non remboursé ce mois	$\Sigma \text{MAX}(0, -\text{ecart_serveuse})$ ce mois
Caissière	<code>sessions_caisse</code> (stocké à la clôture)	$\Sigma \text{MAX}(0, \text{ecart})$ (total esp + OM) ce mois <code>WHERE caissier_id=emp</code>	$\Sigma \text{surplus_caisse}$ ce mois <code>WHERE caissier_id=emp</code>

Dans les deux cas, la déduction nette retransche `SUM(remboursements_ecart.montant WHERE statut=valide ce mois)`.

7. Formules de Calcul — Invariantes

Impératif. Ces formules ne changent jamais. Si une page calcule autrement, **corriger la page** — pas les formules.

7.1 P&L et Distribution

```
-- Rapport journalier :
recettes = total_chicha + total_boissons
net      = recettes - total_achats          → rapports.net

-- Période (finances.html / bilan.html) :
net_for_parts = SUM(rapports.net)
              - SUM(charges.montant)      -- charges ne contient jamais de salaires
              - SUM(salaires_verses.net_verse) -- salaires déduits AVANT distribution
tresorerie    = net_for_parts
              - SUM(avances WHERE statut IN (en_attente, approuvee) AND rembourse=false)

-- Distribution (avec owner_pct +  $\Sigma$  associes.pourcentage = 100) :
distribPct   = MAX(0, 100 - part_lounge%) / 100
owner_pct    = config.owner_pct
part_owner   = MAX(0, net_for_parts) × distribPct × owner_pct / 100
part_assoc_X = MAX(0, net_for_parts) × distribPct × assocX.pourcentage / 100
```

La trésorerie retient les avances `en_attente` **et** `approuvee` (vue prudente) ; la paie ne déduit que les avances `approuvee` (§7.2). Cette différence est volontaire.

7.2 Salaire Mensuel

```
brut      = employees.salaire_base
absNJ     = COUNT(presences.statut=absent ce mois)
          - COUNT(justifications.type=absence AND statut=approuvee ce mois)
sanc_abs  = brut × 0.10 si absNJ == 2
          = brut × 0.15 si absNJ ≥ 3
          = 0          si absNJ < 2
sanc_ret  = brut × 0.10 si retards ≥ 5 ce mois, sinon 0

avances_ded = SUM(avances WHERE statut=approuvee AND rembourse=false) -- TOUTES DATES

-- Écarts (selon le rôle, voir §6.3) – toujours en valeur positive :
ecarts_ded  = (dette du mois) - SUM(remboursements_ecart.montant WHERE statut=valide ce mois)
surplus_bonus = (surplus du mois) -- valeur POSITIVE → augmente le salaire

net_verse = MAX(0, brut - sanc_abs - sanc_ret - avances_ded - ecarts_ded + surplus_bonus)

VERSER → INSERT salaires_verses
        + UPDATE avances SET rembourse=true (avances déduites ce mois pour cet employé)
ANNULLER → DELETE salaires_verses
          + UPDATE avances SET rembourse=false (réintègre les avances de ce versement)
```

ANNULER réintègre les avances (*rembourse=false*) afin qu'elles redeviennent déductibles : aucune avance n'est perdue.

7.3 Bilan

Calcul	Formule	Usage
marge	SUM(rapports.net) (= recettes – achats sur la période)	Marge brute d'exploitation
net_for_parts / resultatNet	marge – SUM(charges.montant) – SUM(salaires.verses.net_verse)	Résultat net = base de distribution des parts
Parts calculées	MAX(0, net_for_parts) × distribPct × pourcentage / 100	Cohérent avec finances.html

net_for_parts et *resultatNet* sont désormais **identiques** : la base de distribution est le résultat net réel (après charges et salaires). Plus de double définition.

8. `shared.js` — Fonctions Disponibles

Fonction	Rôle
<code>ML.getRole()</code>	Rôle connecté (string)
<code>ML.getExtra()</code>	<code>{ nom, employe_id associe_id, pourcentage }</code>
<code>ML.guard(roles[])</code>	Redirige vers <code>index.html</code> si rôle non autorisé
<code>ML.lock()</code>	Déconnecte + redirige <code>index.html</code>
<code>ML.logAction(desc)</code>	INSERT dans <code>logs</code>
<code>ML.initAutoLock()</code>	Démarre le timer auto-lock 10 min
<code>initPage(roles[])</code>	<code>guard + renderNav + initAutoLock + loadNavBadges</code> — appeler au début de chaque page
<code>renderNav()</code>	Génère la nav filtrée par rôle
<code>loadNavBadges()</code>	Charge les points rouges (remboursements + avances)
<code>gnf(n)</code>	Formate un montant en GNF (séparateur de milliers)
<code>frDate(d)</code>	Date longue en français (jour, mois, année)
<code>frDateShort(d)</code>	Date courte (jour + mois)
<code>frMonth(d)</code>	Mois + année
<code>todayISO()</code>	Date du jour au format <code>YYYY-MM-DD</code>
<code>sha256(str)</code>	async → hash hex SHA-256
<code>escHtml(s)</code>	Échappe HTML pour <code>innerHTML</code> — obligatoire anti-XSS
<code>jsStr(s)</code>	Échappe pour <code>onclick="fn(' ... ')"</code> — obligatoire anti-XSS
<code>toast(msg, type)</code>	<code>type</code> = vide / "ok" / "ko"
<code>\$(id)</code>	<code>document.getElementById(id)</code>
<code>HEADER_HTML</code>	En-tête à injecter dans chaque page
<code>db</code>	Instance Supabase (<code>supabase.createClient</code>)

9. Structure de Page & Patterns Supabase

9.1 Structure standard

```
<!DOCTYPE html>
<html lang="fr"><head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>NomPage - Medellin Lounge</title>
  <link rel="stylesheet" href="shared.css">
</head><body>
  <div id="toast"></div>
  <script src="https://cdn.jsdelivr.net/npm/@supabase/supabase-js@2/dist/umd/supabase.min.js"></script>
  <script src="shared.js"></script>
  <script>
    document.body.insertAdjacentHTML("afterbegin", HEADER_HTML);
    if (!initPage(["role1", "role2"])) throw new Error();
    async function init() { /* ... */ }
    init();
  </script>
</body></html>
```

9.2 Patterns critiques

```
// Filtre produits actifs :
.not("actif", "is", false) // OK - couvre false ET null
// .eq("actif", true) // INTERDIT - exclut les NULL

// Création session (atomique - gère UNIQUE date) :
let {data, error} = await db.from("sessions_caisse")
  .insert({date: todayISO()}).select().single();
if (error?.code === "23505") {
  ({data} = await db.from("sessions_caisse")
    .select().eq("date", todayISO()).single());
}

// UPSERT verifications_staff :
await db.from("verifications_staff").upsert(
  {session_id, employe_id, recu_especes, recu_om},
  {onConflict: "session_id, employe_id"}
);

// Ouverture table (si pas encore ouverte) :
const {data: td, error: tderr} = await db.from("tables_lounge")
  .select("ouverte_par").eq("label", tableLabel).single();
if (!tderr && td && !td.ouverte_par) {
  await db.from("tables_lounge")
    .update({ouverte_par: empId}).eq("label", tableLabel);
}

// Reset tables après validation session :
await db.from("tables_lounge").update({ouverte_par: null}).eq("actif", true);

// Fermeture forcée d'une table (owner/manager) :
await db.from("tables_lounge").update({ouverte_par: null}).eq("label", tableLabel);

// Numéro de rapport (MAX+1) :
const {data} = await db.from("rapports")
  .select("num").order("num", {ascending: false}).limit(1);
const nextNum = (data?.[0]?.num ?? 0) + 1;

// Realtime :
const sub = db.channel("nom").on("postgres_changes",
  {event: "*", schema: "public", table: "sessions_caisse"},
  payload => { /* handler */ }
).subscribe();
// Cleanup : db.removeChannel(sub)
```

10. Directives de Code — Non Négociables

10.1 Sécurité — Anti-XSS

- `escHtml()` : toute donnée utilisateur injectée via `innerHTML` passe par `escHtml()` . **Sans exception** — noms, notes, produits, motifs, titres.
- `jsStr()` : toute donnée passée dans `onclick="fn('... ')"` passe par `jsStr()` . Les noms avec apostrophes (N'Diaye, l'eau) cassent le handler sinon.

10.2 Noms de colonnes exacts

```
charges → "label"      PAS "libelle"
produits → "prix_defaut" PAS "prix"
produits → "stock_min"  PAS "seuil_bas"
ventes → "table_label"  PAS "table_num" (TEXT, pas INT)
filtre actifs → .not("actif","is",false) PAS .eq("actif",true)
```

10.3 Règles métier inviolables

1. `avances_ded` : `statut=approuvee AND rembourse=false` — **toutes dates**.
2. `openValidation` → `sessInfo.ecart` stocké, **ne pas recalculer**.
3. `rapport.html` → bloquer si un rapport existe déjà pour cette date (contrainte DB + garde applicative).
4. `avance.html` → bloquer si une demande `en_attente` existe déjà.
5. `saisie.html` → multi-tours ; verrou si `statut ≥ valide_caissier` .
6. `table_label` → trier avant concaténation (ordre alphabétique stable).
7. Ouverture table → `SET ouverte_par=emp_id` si `ouverte_par IS NULL` uniquement.
8. Reset tables → `SET ouverte_par=NULL WHERE actif=true` après `valide_manager` .
9. Fermeture forcée (owner/manager) → `SET ouverte_par=NULL WHERE label=...` .
10. `caissier_id` **obligatoire** à la clôture (sélection si owner/manager clôture).
11. VERSER → `INSERT salaires_verses` + `UPDATE avances SET rembourse=true` .
12. ANNULER → `DELETE salaires_verses` + `UPDATE avances SET rembourse=false` (réintègre les avances).
13. `associe` → lecture seule partout ; ne voit **que sa propre part** (`bilan.html` , `finances.html` , `associes.html`).
14. `sorties_chicha` → stock uniquement, jamais dans les calculs financiers.
15. `rapport.html` → CA pré-rempli depuis `ventes_session` , ajustable ; écart `saisi - calculé` affiché.
16. `total_achats` = `SUM(achats_session)` — non modifiable.
17. L'écart **OM compte** dans la paie de la caissière : déduction = **écart total** (`ecart_especes` + `ecart_om`).
18. `surplus_bonus` est une **valeur positive** (il augmente le salaire).
19. Paiement = **espèces ou OM** uniquement (pas de crédit / ardoise).
20. `owner_pct` + `SUM(associes.pourcentage actifs)` = 100 — vérifié dans `parametres.html` .
21. Les salaires ne sont **jamais** saisis dans `charges` (uniquement `salaires_verses`).
22. Montants : `if(!(montant > 0))` partout — jamais de montant nul ou négatif.
23. Repli paie : `ecart_especes ?? ecart` (sessions antérieures à la migration).

10.4 Suppression de rapport — owner depuis `historique.html`

```
rapport du JOUR → sessions_caisse SET statut="ouvert"
                  (saisie.html se déverrouille - la serveuse peut ressaisir)
rapport PASSÉ → sessions_caisse SET statut="valide_caissier"
                  (revalidation possible depuis dashboard.html)
```

10.5 Realtime — Listeners par page

Page	Table(s) écoutée(s)	Action déclenchée
<code>saisie.html</code>	<code>sessions_caisse</code>	Verrouiller si <code>statut = valide_caissier</code> ou <code>valide_manager</code>
<code>caisse.html</code>	<code>ventes_session</code>	Rafraîchir les cartes serveuses + recalculer les écarts affichés
<code>dashboard.html</code>	<code>sessions_caisse</code> , <code>rapports</code>	Alerter si une session passe à <code>valide_caissier</code> ; recharger les KPIs sur nouveau rapport

11. Pages à Onglets & Comportements Spécifiques

11.1 `rh.html` — 5 onglets (autorité unique présences + avances)

Onglet	Fonctionnalité
Équipe	Liste du staff actif (<code>employees</code> avec <code>actif=true</code>)
Présences	Sélecteur de date, 4 statuts (present/absent/retard/conge), bouton « Tous présents ». Source des présences
Demandes	Avances + justifications <code>en_attente</code> → Approuver / Rejeter
Avances	Le manager ajoute une avance → <code>statut=approuvee</code> directement (sans validation)
Paie	Mois sélectionnable, VERSER par employé (déductions auto), ANNULER

`pointage.html` → `rh.html` > *Présences* ; `avances.html` → `rh.html` > *Demandes/Avances* .

11.2 `finances.html` — accès par rôle

Onglet	Formule / Contenu	Accès
Bilan	<code>net_for_parts = SUM(rapports.net) - SUM(charges.montant) - SUM(salaires_verses.net_verse)</code> — période sélectionnée	owner + manager
Dividendes	Même base — répartition owner + associés	owner uniquement
Trésorerie	<code>net_for_parts - SUM(avances non remboursées)</code>	owner + manager
Charges	Liste des charges par mois (hors salaires)	owner + manager
+ Charge	Formulaire ajout / modification de charge	owner + manager
Évolution	Graphique <code>net_for_parts</code> sur 6 à 18 mois	owner + manager
Mes Parts	<code>MAX(0, net_for_parts) × distribPct × pourcentage / 100</code> sur 6 mois	associé uniquement (sa part)

11.3 `parametres.html`

- **PINS** : owner / manager de secours / staff global (non utilisé en auth — à changer quand même).
- **Config** : `owner_nom` , `message_manager` , `objectif_journalier` , `part_lounge` , `owner_pct` .
- **Validation** : refuse l'enregistrement si `owner_pct + SUM(associes.pourcentage actifs) ≠ 100` .
- **Tables du lounge** : ajouter / renommer / désactiver (`tables_lounge`) — `label` + `ordre` + `actif` .
- **Pas** de reset en masse — suppression rapport par rapport depuis `historique.html` .

12. Ordre de Développement

Étape	Fichier(s)	Note
0	<code>SCHEMA.sql</code>	Exécuter dans Supabase (voir §4b) : créer <code>associes</code> , <code>tables_lounge</code> , <code>verifications_staff</code> ; ajouter <code>ecart_om</code> + <code>surplus_caisse</code> à <code>sessions_caisse</code> , <code>table_label</code> à <code>ventes_session</code> , contrainte UNIQUE sur <code>rapports.date</code>
1	<code>shared.js</code> + <code>shared.css</code>	Fondation — auth, utils, nav, <code>loadNavBadges</code>
2	<code>index.html</code>	Auth 4 étapes (owner / manager secours / employés / associés)
3	<code>saisie.html</code>	Grille tables + <code>ouverte_par</code> + multi-tours + verrou session
4	<code>caisse.html</code>	<code>verifications_staff</code> par serveuse + écarts 3 colonnes + <code>caissier_id</code> obligatoire + vue par table
5	<code>rapport.html</code>	CA auto depuis ventes (ajustable) + <code>total_achats</code> figé + reset <code>ouverte_par</code> + Realtime
6	<code>dashboard.html</code>	KPIs + validation session + Realtime (<code>sessions_caisse</code> + <code>rapports</code>)
7	<code>parametres.html</code>	PINs + config + invariant des % + gestion <code>tables_lounge</code>
8	<code>rh.html</code>	5 onglets + formule salaire complète (serveuse + caissière)
9	<code>produits.html</code> , <code>pointage.html</code> , <code>avances.html</code> , <code>charges.html</code>	Stock + raccourcis RH + charges hors salaires
10	<code>finances.html</code> , <code>bilan.html</code> , <code>associes.html</code>	Finances et parts (salaires déduits, confidentialité associé)
11	<code>historique.html</code> , <code>fiche.html</code> , <code>avance.html</code> , <code>chicha.html</code>	Compléments

13. Prompt Maître pour Claude Code

À copier-coller intégralement au début de chaque session Claude Code. Mettre à jour la section **ÉTAPE EN COURS** à chaque session.

```

# MEDELLIN LOUNGE - CONTEXTE COMPLET
# Coller ce bloc INTÉGRALEMENT au début de chaque session Claude Code.

## STACK
HTML/CSS/JS vanilla · Supabase JS v2 (CDN) · Netlify Pro
Dépôt : https://github.com/jeunesavane-ctrl/Monprojet.git (main)
Prod : https://medellin-lounge.com
Local : python -m http.server 5500
DEPLOY : git push = versioning UNIQUEMENT. Deploy = manuel Netlify.
JAMAIS dire « c'est en ligne » après un push.

## SUPABASE
URL : https://xkdlkvwtzfixsaiexdkf.supabase.co
Key : sb_publishable_w_H1J0lysnd1KFIs3bmQlg_KMbsyc44
RLS : DÉSACTIVÉ - voir SECURITE.md pour le plan de bascule

## 3 CATÉGORIES D'ACTEURS - EMPLACEMENTS SÉPARÉS
owner → config (pin_owner, owner_nom, owner_pct)
associe → associes (id, nom, prenom, pourcentage, pin_hash, actif)
employes → employes (id, nom, prenom, poste, role, salaire_base, pin_hash, actif)
roles employes : manager / caissier / staff / chicha / achats
Le manager est un EMPLOYÉ suivi en paie. config.pin_manager = accès de secours (sans employe_id).

## AUTH - ORDRE STRICT dans index.html
1. sha256(PIN) === config.pin_owner → "owner" extra={nom: owner_nom}
2. sha256(PIN) === config.pin_manager → "manager" extra={nom:"Manager", acces:"secours"}
3. sha256(PIN) === employes.pin_hash → emp.role extra={nom, employe_id}
4. sha256(PIN) === associes.pin_hash → "associe" extra={nom, associe_id, pourcentage}
5. aucune correspondance → erreur, pas de connexion

## 22 TABLES - COLONNES EXACTES
config : key TEXT UNIQUE, value TEXT
associes : id, nom, prenom, pourcentage, pin_hash, actif
employes : id, nom, prenom, poste, role, salaire_base, pin_hash, actif
sessions_caisse : id, date UNIQUE, statut (ouvert/valide-caissier/valide_manager),
fond_caisse, total_reel, total_om_verifie, ecart_especes, ecart_om, ecart,
surplus_caisse, caissier_id (NULL à la création, obligatoire à la clôture), note_caissier, note_manager
ventes_session : id, session_id, employe_id, produit_id, produit_nom,
qty, prix_unitaire, total, paiement (especes/om), table_label TEXT
verifications_staff : id, session_id, employe_id, recu_especes, recu_om
UNIQUE(session_id, employe_id)
tables_lounge : id, label TEXT UNIQUE, ordre INT, actif BOOLEAN,
ouverte_par UUID (nullable → employes)
produits : id, nom, type, stock_actuel, stock_min, prix_default, prix_achat,
unite_vente, packaging_label, unite_par_packaging, actif NOT NULL DEFAULT true
charges : id, label (PAS libelle), montant, mois (YYYY-MM), categorie,
paye, date_paiement, recurrence -- JAMAIS de salaire
avances : id, employe_id, montant, date, statut, rembourser, note_demande, obs
salaires_verses : UNIQUE(employe_id, mois) - ... net_verse, surplus_caisse ...
presences : UNIQUE(employe_id, date) - statut : present/absent/retard/conge
justifications : id, employe_id, date, type (absence/retard/autre), motif, statut
remboursements_ecart : id, session_id, employe_id, montant, statut
rapports : id, date UNIQUE, num INT, session_id, total_chicha, total_boissons,
total_achats, recettes, net
mouvements_caisse : id, session_id, type (entree/sortie), motif, montant, note
achats_session : id, session_id, categorie, produit_nom, montant, qty, prix_unitaire
sorties_chicha : id, session_id, employe_id, arôme, qty, valide
credits : id, employe_id, session_id, montant, rembourser
propositions : id, titre, description, auteur_nom, statut
votes_prop : id, proposition_id, votant_key, votant_nom, poids, choix
UNIQUE(proposition_id, votant_key)
logs : id, role, action, timestamp

## FORMULES INVARIANTES
-- CA journalier (rapport.html, pré-rempli depuis ventes_session par type, ajustable) :
total_chicha = SUM(ventes_session.total WHERE produit.type=chicha)
total_boissons = SUM(ventes_session.total WHERE produit.type IN (boisson, autre))
total_achats = SUM(achats_session.montant) -- non modifiable
recettes = total_chicha + total_boissons ; net = recettes - total_achats
-- Écart caissière (validateSession) :
theoriqueEsp = fond + SUM(verif_staff.recu_especes) + entrees - sorties - achats
ecart_especes = theoriqueEsp - total_reel
ecart_om = SUM(verif_staff.recu_om) - total_om_verifie
ecart = ecart_especes + ecart_om
surplus_caisse = MAX(0, -ecart)
-- Écart serveuse (JS, jamais stocké) :

```

```

ecart_serv = (decl_esp + decl_om) - (recu_esp + recu_om)
-- Salaire :
net_verse = MAX(0, brut - sanc_abs - sanc_ret - avances_ded - ecarts_ded + surplus_bonus)
surplus_bonus est POSITIF. avances_ded : approuvee AND rembourse=false - TOUTES DATES.
Paie caissière : écart TOTAL (ecart_especes + ecart_om) - l'OM compte (téléphone chez la caissière).
-- Distribution (owner_pct + Σ% = 100) :
net_for_parts = SUM(rapports.net) - SUM(charges.montant) - SUM(salaires_verses.net_verse)
distribuable = MAX(0, net_for_parts) × (100 - part_lounge%) / 100
part_owner = distribuable × owner_pct / 100
part_assoc_X = distribuable × assocX.pourcentage / 100

## RÈGLES INVOLABLES
1. innerHTML → escHtml() SANS EXCEPTION
2. onclick avec données → jsStr()
3. Produits actifs → .not("actif","is",false) JAMAIS .eq("actif",true)
4. charges → "label" (JAMAIS "libelle") ; JAMAIS de salaire dans charges
5. produits → "prix_defaut" (PAS "prix"), "stock_min" (PAS "seuil_bas")
6. ventes → "table_label" TEXT (PAS "table_num" INT)
7. paiement = especes ou om uniquement (pas de crédit / ardoise)
8. tables_lounge.ouverte_par = UUID employé OU NULL ; ouverture si IS NULL uniquement
9. Reset tables → ouverte_par=NULL WHERE actif=true après valide_manager
10. Fermeture forcée → ouverte_par=NULL WHERE label=X (owner/manager)
11. caissier_id obligatoire à la clôture
12. openValidation → sessInfo.ecart stocké - NE PAS recalculer
13. rapport.html → 1 rapport/date (UNIQUE DB) ; total_achats figé ; CA ajustable
14. avance.html → bloquer si demande en_attente existe déjà
15. saisie.html → multi-tours ; verrouiller si statut ≥ valide_caissier
16. VERSER → INSERT salaires_verses + UPDATE avances SET rembourse=true
17. ANNULER → DELETE salaires_verses + UPDATE avances SET rembourse=false
18. associe → lecture seule, voit uniquement SA part
19. sorties_chicha → stock uniquement, jamais financier
20. écart OM compté dans la paie caissière (écart TOTAL) ; surplus_bonus est positif
21. owner_pct + Σ associes.pourcentage = 100 (validé parametres.html)
22. salaires déduits de net_for_parts AVANT distribution
23. montants : if(!montant>0)) partout
24. suppression rapport jour → session statut="ouvert" ; passé → "valide_caissier"
25. deploy = manuel Netlify - jamais « c'est en ligne » après push

## PATTERNS SUPABASE
-- Session atomique :
let {data,err} = await db.from("sessions_caisse")
  .insert({date:todayISO()}).select().single();
if (err?.code==="23505") ({data} = await db.from("sessions_caisse")
  .select().eq("date",todayISO()).single());
-- UPSERT verifications_staff :
await db.from("verifications_staff").upsert(
  {session_id,employe_id,recu_especes,recu_om},
  {onConflict:"session_id,employe_id"});
-- Ouverture table (si libre) :
const {data:td} = await db.from("tables_lounge")
  .select("ouverte_par").eq("label",tableLabel).single();
if (td && !td.ouverte_par) await db.from("tables_lounge")
  .update({ouverte_par:empId}).eq("label",tableLabel);
-- Reset tables après valide_manager :
await db.from("tables_lounge").update({ouverte_par:null}).eq("actif",true);
-- Numéro rapport :
const {data} = await db.from("rapports")
  .select("num").order("num",{ascending:false}).limit(1);
const nextNum = (data?.[0]?.num ?? 0) + 1;

## ÉTAPE EN COURS - METTRE À JOUR À CHAQUE SESSION
ÉTAPE : [numéro et nom]
FICHER : [fichier(s) à modifier]
OBJECTIF : [ce qui doit être fait précisément]
AVANT : [vérifications à faire avant de commencer]

```

14. Règles de Conduite — Non Négociables

1. **Ce document fait autorité.** En cas de doute, suivre ce document — corriger le code, pas le document.
2. **Une étape à la fois.** Terminée + testée sur `localhost:5500` + poussée sur git avant de passer à la suivante.
3. `escHtml()` ET `jsStr()` sans exception. Aucun raccourci possible sur la sécurité XSS.

4. **Associés ≠ Employés.** Emplacements strictement séparés. Jamais stocker un associé dans `employees`.
 5. `verifications_staff` **est la base des écarts.** `theorique = fond + ce que la caissière a reçu de chaque serveuse`, pas directement les ventes.
 6. **Le CA est dérivé des ventes.** `rapport.html` pré-remplit depuis `ventes_session` ; tout ajustement manuel affiche un écart visible.
 7. **Les salaires sont déduits avant la distribution** et ne figurent jamais dans `charges`.
 8. **Les formules ne changent pas.** Si une page calcule autrement, corriger la page.
 9. `table_label` **est un TEXT.** Pas un entier. Trier avant concaténation. Pas de `table_num`.
 10. `sorties_chicha` = **stock uniquement.** Jamais dans les calculs financiers.
 11. **Deploy = manuel Netlify.** `git push` = versioning. Jamais « c'est en ligne » après un push.
 12. `SECURITE.md` **est la référence sécurité.** RLS + Edge Function planifiés. Ne pas improviser côté HTML.
-